Diabetes Prediction System tools primer

Updated 11 November 2026. Start with named data, then trace what each tool contributes to the final score.

1. Python, environments and notebooks

Python runs the training and scoring commands. A virtual environment keeps dependencies for this project separate from other repositories. The requirements file pins tested package versions. The notebook provides an interactive path through the same Python code.

> A notebook is a lab bench. The script is the repeatable procedure you can run after clearing the bench.

2. CSV, pandas and feature names

A CSV is text organised into rows and columns. The previous file extension suggested an Excel workbook, but its contents were CSV. Pandas reads the header and values into a DataFrame. The loader checks that the named columns match the schema before model fitting.

At inference, a JSON record maps names to values. The code selects fields in training order. If age and glucose swap positions in an unnamed list, a model can score the wrong meaning without a shape error.

3. Missing values and imputation

Median imputation replaces a missing value with the middle observed training value. For observed values 70, 82 and 110, the median is 82. The five zero-coded measurements are mapped to missing values before computing medians. A zero pregnancy count remains in the data.

A missing-value flag can sometimes help a model learn when absence carries information. The current pipeline uses median imputation and evaluates its actual effect.

4. Splitting and leakage

The training partition teaches the model. The holdout estimates performance on separate records from the benchmark. Stratification keeps class proportions close across the split.

Leakage occurs if a step learns from examples meant for evaluation. Computing a median or mean across all rows exposes held-out feature values to training. A scikit-learn Pipeline fits the imputer and scaler within each training fold and applies those same fitted steps to that fold's validation rows.

5. Standardisation and a linear SVM

Standardisation uses the training formula `z = (x - mean) / standard_deviation`. If a training mean is 100 and its standard deviation is 20, a value of 120 maps to 1. The model's linear score adds a weighted contribution from each scaled feature.

A support vector classifier seeks a wide margin while penalising violations. The parameter C controls that penalty. This project uses a fixed linear model and default C. Tune C on development data before evaluating on a fresh holdout.

6. Metrics you can explain aloud

Metric	Calculation on the holdout	Result
Accuracy	119 correct / 154 records	77.27%
Precision	28 true positives / 37 positive predictions	75.68%
Recall	28 true positives / 54 actual positives	51.85%
Specificity	91 true negatives / 100 actual negatives	91.00%
Balanced accuracy	Average of recall and specificity	71.43%

ROC AUC summarises how well positive records rank above negative records as the threshold varies. It is 0.8200 for the repaired classifier. Accuracy and recall use one threshold, so they can stay constant while AUC changes.

7. Cross-validation and honest uncertainty

Five-fold validation rotates the validation partition inside the training data. It provides five estimates rather than a single training split. Here, accuracy averaged 77.04% and varied by 1.36 percentage points in standard deviation. This standard deviation describes fold variation. It does not measure how the model performs at a different hospital or time.

8. Persistence and reproducibility

Joblib saves the fitted scikit-learn pipeline. Reloading it skips retraining and reuses the medians, scaler parameters and learned SVC. A dataset digest identifies the exact CSV bytes used in a run. Use the pinned environment with trusted artifacts.

Practise on the example

Run `python diabetes.py train` and `python diabetes.py predict examples/record.json`. In a copy of the example JSON, set Insulin to null and rerun prediction. Then set Age to a negative number and check the error. Explain why the first record can be imputed and the second is rejected.

Sources: Run instructions | Execution audit | Dataset reference